```
        .isEqualTo(new Book("1234567890", "Title", "Author", 9.90));
    }
}
```

❶ Identifies a test class that focuses on JSON serialization

❷ Utility class to assert JSON serialization and deserialization

❸ Verifying the parsing from Java to JSON, using the JsonPath format to navigate the JSON object

❹ Defines a JSON object using the Java text block feature

❺ Verifies the parsing from JSON to Java

WARNING If you use IntelliJ IDEA, you might get a warning that `JacksonTester` cannot be autowired. Don't worry. It's a false positive. You can get rid of the warning by annotating the field with `@SuppressWarnings("SpringJavaInjectionPointsAutowiringInspection")`.

You can run the tests with the following command:

```
$ ./gradlew test --tests BookJsonTests
```

In the code repository accompanying the book, you can find more examples of unit and integration tests for the Catalog Service project.

After automating the tests for your application, it's time to automate its execution whenever a new feature or bug fix is delivered. The following section will introduce the key pattern of continuous delivery: the deployment pipeline.

3.5 Deployment pipeline: Build and test

Continuous delivery is a holistic approach for quickly, reliably, and safely delivering high-quality software, as I explained in chapter 1. The primary pattern for adopting such an approach is the deployment pipeline, which